

Contents

ImageCAS-X — how this repository works	2
0. What this repository is	2
1. The data	3
2. Config: how a run is specified	4
3. The three stages, and why they are separate	5
4. The four registries	7
5. Dataloading in detail	8
6. The eight methods	8
7. Losses	10
8. Metrics and evaluation	11
9. The centerline analysis code	12
10. The offline-precompute pattern	13
11. Running things end to end	14
12. Things to know before you change anything	15

ImageCAS-X — how this repository works

A walkthrough of the codebase: what each piece does, how the pieces fit together, and the design decisions that explain why it looks the way it does.

0. What this repository is

ImageCAS-X is a *benchmark*, not a single model. It is the code accompanying Bransby, Øksnebjerg, Kjær, Kirkeby, El Youssef, Jiménez, Pedersson, de Knecht, Kofoed & Paulsen (2026) — a re-annotation of the 800 CCTA scans in the public ImageCAS cohort with new labels (coronary lumen split into 14 named segments, centerlines, and surface meshes), plus reimplementations of eight segmentation methods trained and scored under one fixed protocol.

The whole design follows from one goal: **make eight very different methods comparable**. Everything that could bias a comparison — preprocessing, augmentation, optimizer, schedule, what counts as an epoch, postprocessing, metrics — is pulled out of the methods and fixed in one shared config. What is left in a method's own config is only what makes that method that method.

The codebase has three top-level entry points (`train.py`, `inference.py`, `evaluate.py`) and eight packages. There is no framework beyond PyTorch; the plumbing is hand-rolled and small enough to read end to end.

```
ImageCAS-X/
├── train.py  inference.py  evaluate.py      ← the three stages
├── configs/                                     ← one JSON per method + pipeline.json
├── utils/config.py                             ← config loading and merging
├── dataloading/                               ← datasets, keyed by input_type
├── preprocessing/  augmentation/  postprocessing/ ← step pipelines
├── models/                                               ← architectures + registry
├── losses/                                               ← loss functions + registry
├── utils/                                               ← I/O, metrics, offline precomputes
└── docs/                                               ← the project website
```

1. The data

On disk

Everything hangs off two environment variables (utils/config.py:9-10):

```
export ImageCAS_X_data_path=/path/to/data
export ImageCAS_X_results_path=/path/to/results
```

These **override** whatever the JSON says (utils/config.py:135-140), which is why the configs in the repo can be checked in without any machine-specific paths.

The expected layout (README.MD:106-113):

```
<data_root>/
├── volumes/           <scan_id>.img.nii.gz           CCTA volume
├── segmentations/     <scan_id>.coronary.nii.gz           multi-label mask, 0-14
├── centerlines/       <scan_id>.coronary_{left,right}_centerline.vtk
├── surfaces/          <scan_id>.coronary_surface.vtk           (see note below –
should be .coronary_mesh.vtk)
├── filelist/          train.txt val.txt test.txt exclude.txt
└── Descriptors.xlsx   Scan ID | Image Quality | Dominance | Disease
```

Every directory name and suffix in that tree is a config field (data.volume_dir, data.mask_suffix, ...), so a different layout is a config change, not a code change.

The label scheme

Masks are **multi-label**, not binary (README.MD:42-58, mirrored as SEGMENT_NAMES in evaluate.py:256-260):

0	background	5	D2	10	R-PDA
1	LM	6	OM1	11	R-PLA
2	LAD	7	OM2	12	L-PDA
3	LCx	8	IM	13	L-PLA
4	D1	9	RCA	14	Other (D3, D4, OM3, OM4)

The default pipeline **binarises** this to lumen-vs-background in the dataloader (bio.binarise_lumen, utils/io.py:25; toggle with data.params.binarise_lumen), because all eight benchmarked methods do binary lumen segmentation. The segment labels are still used at evaluation time for per-segment breakdowns, and evaluate.py --multi_class scores all 14 labels individually.

Note that labels 10/11 are the *right*-sided posterior descending and posterolateral arteries and 12/13 are the *left*-sided ones. Coronary dominance is therefore encoded in the labels themselves, in addition to being a column in Descriptors.xlsx.

The centerline VTKs

These are richer than they look. Each file is VTK polydata carrying:

- **points** — (N,3) in LPS millimetres of the *original* scan (not the resampled grid).
- **line cells** — polylines that index into the one shared point array.
- **point-data arrays** — segment_label (1-14), segment_name (strings), start_points (os-tia), end_points (tips), branch_points (bifurcations).

Because all cells index one shared point array, **a point ID is already a graph node**, and two cells meeting at a bifurcation join automatically. This is the basis of all the centerline analysis

in the repo.

One trap, documented at `utils/precompute_centerline_samples.py:11-13`: the line cells have **no consistent proximal→distal orientation**. Cell order tells you nothing about which end is closer to the ostium. Direction must come from a traversal rooted at `start_points`.

Offline caches

Every method wants the volume resampled to 0.5 mm isotropic. Doing that inside `__getitem__` would dominate training time, so it is done once, up front:

```
python -m utils.offline_resample_images_to_disk -c configs/<method>.json --workers 8
```

This writes `.npy` arrays to `volumes_resampled/` and `segmentations_resampled/` (masks stay multi-label). A second cache, `volumes_resampled_128/`, holds the $128 \times 128 \times 64$ downsampled volumes the coarse stages use, plus a `_spacing.json` sidecar recording each scan's resulting anisotropic spacing.

The dataloader knows about these caches and, when it uses one, **skips the preprocessing steps already baked into it** (`dataloading/base_dataset.py:19-22, :127`). So `preprocessing.steps` in the config is still the honest description of what happened to the data, even though at runtime some of those steps are no-ops. Same trick for dilated masks: `<scan_id>_dilated.npy` siblings are picked up automatically when `dilate_mask` is in the step list, because dilating a radius-2 ball over a 0.5 mm volume costs ~ 7 s per sample.

2. Config: how a run is specified

`utils/config.py` is the spine. Two ideas:

1. Every method config is merged over `configs/pipeline.json`. `BenchmarkConfig.from_json(:180)` loads `configs/pipeline.json`, then deep-merges the method's JSON on top, method keys winning (`_deep_merge, :19`). Nested dicts merge key-by-key; **lists are replaced outright**, so declaring `preprocessing.steps` in a method config replaces the whole list rather than appending to it.

`pipeline.json` is the fairness contract. It fixes:

Preprocessing	resample to 0.5 mm iso; HU window $[-200, 1000] \rightarrow [0, 1]$
Augmentation	the 8 nnU-Net defaults (flip, noise, blur, brightness, contrast, low-res simulation, $\gamma \pm$)
Training	adam, weight decay $1e-8$, poly LR (power 0.9), 1000 epochs, 250 train iters and 50 val iters per epoch, 14 workers, best-checkpoint on dice
Postprocessing	threshold at 0.5, then drop connected components under 100 voxels
Evaluation	the 11 metrics, the three stratification columns, and the point-metric bins

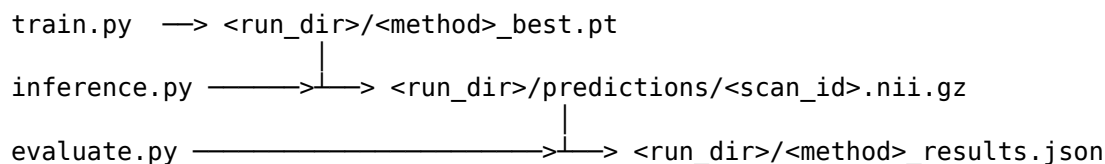
A method config then declares only `method_name`, `input_type`, `model`, `loss`, `batch_size`, `epochs`, and any deliberate override. `configs/cas_net.json` is 39 lines.

2. Config is a set of dataclasses, filtered leniently. DataConfig, ModelConfig, LossConfig, TrainingConfig, PreprocessingConfig, PostprocessingConfig, EvaluationConfig (utils/config.py:31-122). `_pick(:13)` drops unknown keys rather than raising, so comments and extra fields in a JSON are harmless. augmentation stays a raw dict (:150).

Splits come from `<filelist_dir>/{train,val,test}.txt`, one scan ID per line, and IDs in `exclude.txt` are dropped from every split (`_load_filelist_ids, :162`). Filelists take precedence over inline `train_ids/val_ids/test_ids`.

Some configs have no model at all. `nnunet.json`, `nnunet_cldice.json`, `total_segmentator.json`, `inter_observer.json`, `imagecas_manual.json` and `imagecas_ensemble.json` name a `model.name` that is *not in the registry*. That is deliberate — they are only ever fed to `evaluate.py`, which never builds a model. This is how externally-produced predictions (and a second human observer) get scored on exactly the same footing as the methods trained in this repo.

3. The three stages, and why they are separate



The separation is load-bearing. `evaluate.py` **never loads a model** (`evaluate.py:8-9`). It scores whatever files sit in `predictions/`. So nnU-Net (a full external training framework), TotalSegmentator (an external pretrained tool), and a second human analyst’s annotations all enter the benchmark by dropping files into that folder with the right names. Nothing special is needed for them.

train.py

`python -m train -c configs/<method>.json — -c` is the only argument. Everything else is config.

The flow (`train()` at :163):

1. `seed_everything()` — SEED 42, `cuda.deterministic=True`, `benchmark=False` (`utils/seeding.py:13`).
2. Build a timestamped run dir `<results_root>/<method_name>_<YYYY-MM-DD-HH-MM-SS-...>` and mirror stdout into `log.txt` (`_Tee, :35`).
3. Build, in order: preprocessing → augmentation (train split only) → train/val dataloaders → model → loss → optimizer → scheduler (:190-199).
4. Loop. Forward under bf16 autocast, then **cast logits back to fp32 before the loss** (`_logits_to_fp32, :129-136`) — bf16’s 8-bit mantissa is too coarse for a Dice sum over $\sim 10^6$ voxels. There is no GradScaler; bf16 does not need one.
5. Validate with a pseudo-Dice, skipping GT patches with no foreground (`_dice_score, :294`). Save the best checkpoint as `<method_name>_best.pt`. Redraw `training_curves.png` every epoch.

Three details worth internalising:

- An “epoch” is a fixed number of iterations, not one pass over the data (`utils/config.py:81-84`). 250 train iterations, 50 val iterations. This is what makes the schedule comparable across methods whose datasets have wildly different lengths (a patch dataset has millions of samples, a volume dataset has ~ 600).

- **_batch_targets (:101) forwards every tensor the dataset produced**, minus a skip-set of known non-targets. So adding a method with new supervision targets needs a dataset and a loss — not an edit to `train.py`.
- **_compute_loss (:147) dispatches three ways**: a list of logits means deep supervision (mean over heads); `loss_fn.expects_dict` means the loss reads the whole output dict (ADE-HTL's four heads); otherwise plain `loss(logits, mask)`.

inference.py

`python -m inference -c configs/<method>.json -r <run_dir> —` `-r` is required and names the run folder `train.py` printed.

The stable contract is one function (`run_inference(model, volume, cfg, device, scan_id), :253`): full preprocessed volume in, full-volume logits on CPU out. Inside, it branches on `input_type`:

- **Patch / crop models** → `patch_stitch_inference (:70)`: tile the volume with 50% overlap, run each tile, and blend with a Gaussian weight map (nnU-Net's `sigma_scale=0.125`) so tile seams do not show.
- **Patch models with centers_dir** → `run_skeleton_patch_inference (:162)`: predict only at precomputed skeleton centres rather than densely.
- **Everything else** → one forward pass over the whole volume.

Wrapped around all three: **mirror test-time augmentation** over X and Y only (`_MIRROR_AXES = (2,3), :28`) — Z is left alone because training only flips axes 0 and 1.

Then per scan (`predict(), :360`): sigmoid → optional connectivity fusion → re-embed any crop into the full volume → resample the probabilities back to the *original* scan geometry → run the postprocessing pipeline → write `<run_dir>/predictions/<scan_id>.nii.gz`. Predictions for non-test splits go to `predictions_<split>/` so they can never overwrite the test set.

Default behaviour is **resume**: scans already on disk are skipped unless `--overwrite`.

evaluate.py

`python -m evaluate -c configs/<method>.json -r <run_dir> —` plus `-j` for worker count and `--multi_class`.

Per scan, in a worker process: load the prediction (which carries the original volume's header), resample the GT onto *the prediction's* grid with nearest-neighbour, then compute metrics. Because both are in the original scan space, all distances are in true millimetres and all volumes in true millilitres.

Each worker rebuilds its own `BenchmarkConfig` from the path (`_evaluate_scan, :125`) so that `BenchmarkConfig` never has to be picklable — a small trick worth noticing if you ever add state to the config.

Output is one JSON, `<run_dir>/<method_name>_results.json`, with:

- `per_scan` — every metric for every scan
- `summary` — dataset-wide
- `coverage` — how many predictions were actually found, and which IDs were missing
- `summary_by_group[column][value]` — stratified by Image Quality, Dominance, Disease
- `summary_by_point_group[covariate][bin]` — the geometric breakdown (below)

plus `point_metrics.csv`.

4. The four registries

Every extension point in the repo is the same shape: a name→class dict, a decorator or direct entry, and a `build_*(config)` factory that reads the name out of the config. Learn one and you know all four.

Models — `models/registry.py`

```
@register_model("my_method")
class MyMethod(BaseLumenModel):
    def forward(self, x):          # x: (B, 1, X, Y, Z)
        return {"logits": self.net(x)}
```

`build_model(cfg) (:22)` instantiates `cls(**cfg.model.params)` and loads weights. Modules are imported at the bottom of the file so the decorators fire (:45).

The contract (`models/base_model.py:6`): models take a (B, C, X, Y, Z) float tensor and return a **dict with at least "logits", raw and unactivated**, so postprocessing is uniform across methods.

Registered: `imagecas_coarse`, `imagecas_patch`, `imagecas_baseline`, `ade_htl`, `cas_net`, `fpr_unet`, `swin_unetr`.

`is_staged_model(name) (:15)` means “the class has a `load_stage_weights` method” — that is how the multi-checkpoint ImageCAS ensemble is distinguished from a single-weights model.

Datasets — `dataloading/factory.py:22`, keyed by `input_type`

input_type	class	what a sample is
volume	VolumeDataset	the whole preprocessed scan
patch	PatchDataset	one fixed-size patch at a precomputed centre
random_crop	RandomCropDataset	a random crop, with a foreground quota
ade_htl_crop	ADEHTLDataset	a crop inside the ADE ROI, with 4 supervision targets

Losses — `losses/factory.py:9`

`dice`, `dice_ce`, `multiclass_dice`, `weighted_sim`, `deep_supervision_dice`, `ade_htl`.

Pipeline steps — `preprocessing/pipeline.py`, `augmentation/pipeline.py`, `postprocessing/pipeline.py`

All three: an ABC with `__init__(**params)` and `__call__(sample) -> sample`, a `_STEP_REGISTRY`, a Pipeline that chains them, and a `build_*(config)` that reads `config.<phase>.steps` and looks up per-step kwargs in `config.<phase>.params[step_name]`.

Adding anything to any of these is: write the class, add one registry entry, name it in a config.

5. Dataloading in detail

`BaseLumenDataset` (`dataloading/base_dataset.py:14`) handles everything common: resolving paths, choosing between the sitk file and the .npy cache, running preprocessing while skipping cache-baked steps, binarising the mask, and loading centerlines. A sample always carries at least volume, mask, `scan_id`, spacing.

The four datasets differ in what a “sample” means:

VolumeDataset — the whole scan. Variable shape, so batch size is forced to 1 unless a `resample_to_shape` step gives every scan the same size.

PatchDataset — the index is built from `<centers_dir>/<scan_id>.npy`, one entry per precomputed patch centre. Optionally retries up to 10 times to avoid empty patches. In cache mode it slices straight out of a memory-mapped array, which is what makes patch training fast.

RandomCropDataset — the subtle one. Its docstring (:18-20) says it outright: *“idx is NOT a scan selector here — it carries a bool from ForegroundQuotaBatchSampler saying whether this batch slot must land on foreground.”* Small crops of a coronary tree almost never contain a vessel under uniform sampling, so the sampler enforces a minimum foreground fraction per batch (default 0.5). Origin search runs on the cheap uint8 mask only (up to 20 tries), and the expensive volume crop happens once at the end.

ADEHTLDataset — subclasses `RandomCropDataset`, restricts crops to the ADE region of interest, and densifies sparse precomputed arrays into the crop. It emits a 6-channel volume (CT + 5 anatomical distance fields) plus connectivity, centerline-heatmap and key-point targets. Crucially, the connectivity target is **derived after augmentation, not precomputed** (:16-18) — flipping the volume permutes the meaning of the 27 neighbour channels, so a precomputed target would silently be wrong.

`build_dataloader` (`factory.py:54`) then wires it up:

- Train and val are **endless** — a fixed `n_iters` batches, via `ForegroundQuotaBatchSampler` for crop datasets or a seeded `RandomSampler(replacement=True)` otherwise.
- Test is one deterministic in-order pass.
- `_collate_fn` (:13) pops centerlines out of the default collate because its arrays are variable-length and hold nested dicts.
- `build_eval_dataloader` (:108) always yields whole volumes at batch size 1, whatever the training `input_type` — inference tiles internally, so metrics always see a full-volume prediction.

6. The eight methods

Five are implemented here; three are external and only evaluated.

Method	Idea in one line
CAS-Net (Dong et al., MedIA 2023)	ResNet encoder → <i>scale-aware</i> dilated-conv bottleneck with anisotropic non-local attention → attention-gated skips → multi-scale head. Outputs 2 channels, binarised by argmax.

Method	Idea in one line
FFR-UNet (Song et al., 2022)	U-Net whose encoder stages are dense feature-fusion blocks and whose decoder inserts a residual block after each skip fusion.
Swin UNETR (Hatamizadeh et al., MICCAI 2022)	Hand-rolled 3D Swin transformer encoder (windowed + shifted-window attention, learned 3D relative position bias) with a convolutional decoder.
ImageCAS (Zeng et al., CMIG 2023)	A 5-checkpoint cascade, see below.
ADE-HTL (Zhang et al., IEEE TMI 2024)	Anatomical priors + multi-task topology supervision, see below.
nnU-Net	Trained with the external nnU-Net package; predictions dropped into predictions/.
nnU-Net + clDice	Same, with a soft-skeleton topology loss (Shit et al., CVPR 2021).
TotalSegmentator	External pretrained general-purpose tool.

ImageCAS — the 3-stage, 5-checkpoint cascade

```

Stage 1  coarse U-Net on 128×128×64, plain GT, dice loss          → votes
Stage 2  same net, DILATED GT, weighted-similarity loss          → never votes
        |
        |→ dilate → drop small components → skeletonise → patch centres
Stage 3  three U-Net++ nets at 163, 323, 643, trained separately → all three vote
        |
Final    majority vote over Stage 1 + the three Stage-3 masks

```

Stage 2 exists purely to *find where the vessels are*, not to segment them — its dilated training target and its weighted_sim loss (losses/common.py:96, ImageCAS Eq. 2 with $a=0.01$) penalise false negatives far more than false positives, so thin distal vessels survive skeletonisation and get a patch centre placed on them. Stage 1, trained on the plain mask, is the one that actually votes.

ImageCASBaseline.forward (models/imagecas_baseline/model.py:305) runs this whole cascade in eval mode; in training mode it returns only the coarse logits so Stage 1 can be fine-tuned with gradients. The final output is deliberately *pseudo-logits* $((\text{vote} \times 2 - 1) \times 20)$ so that the framework's uniform sigmoid → threshold 0.5 postprocessing recovers the vote exactly. Its own docstring calls this hard 4-way vote (Stage 1 + 3 patch scales) the *paper-faithful* reproduction of Zeng et al. 2023 — i.e. faithful to the **original ImageCAS method**, not necessarily how the ImageCAS-X benchmark table itself was produced.

The published ImageCAS-X benchmark row does not use this hard vote. The ImageCAS-X paper's supplementary methods (F.4) state the row in their results table was produced by *ensembling probabilities* rather than majority-voting masks, and by *excluding the Stage 1 coarse model* from the ensemble entirely, because doing so improved performance. That is exactly what ensemble_vote.py implements — a post-hoc soft vote over the three patch-scale probability maps (`_prob.nii.gz`, written by inference.py --save-probs), with Stage 1 optional and, per the paper, best left out. Consistent with this, configs/imagecas_ensemble.json is a separate, minimal evaluate-only config (model.name: "imagecas_ensemble") distinct from imagecas_inference.json's in-repo cascade — it exists to score whatever ensemble_vote.py produced. So: ImageCASBaseline.forward() is kept in the repo as a faithful reimplement of the *original* method, while ensemble_vote.py is

what generated the number actually reported for “ImageCAS” in the ImageCAS-X paper.

The repo also flags a second, smaller deviation from the original paper explicitly. `_remove_small_components (:33-48)` uses a size floor instead of Zeng et al.’s keep-the-two-largest rule, with the reasoning spelled out: *hardcoding how many components to expect discards genuine distal branches, and those are unrecoverable downstream — no centre is placed there, so no patch net ever looks at them.*

ADE-HTL — anatomical priors plus topology supervision

ADE = Anatomical Dependency Encoding, and it is *offline*, not part of the network. `precompute_ade.py` takes TotalSegmentator’s heartchambers_highres masks and builds five distance fields — one each to LV, RV, LA, RA and aorta — clipped at 50 mm and stored sparsely. These become input channels 1-5 alongside the CT, so `in_channels = 6`. The chamber masks are an input at *both* train and test time, because TotalSegmentator’s licence does not permit training new models on its outputs.

HTL = Hierarchical Topology Learning, the network. One shared residual U-Net encoder feeds three decoders:

1. **connectivity** — 27 channels, one per $3\times 3\times 3$ neighbour offset, “is this voxel connected to that neighbour”. Channel 13 (the self channel) is the segmentation.
2. **centerline** — a Gaussian heatmap whose per-point sigma is the local vessel radius.
3. **key points** — bifurcations and endpoints.

They are wired **bottom-up** by attention blocks: connectivity informs centerline, centerline informs key points (`model.py:262`).

At inference, `connectivity_votes (:41)` implements the paper’s pairwise-consistency rule: two voxels only agree if *each* claims the other, and the self channel both seeds and gates the vote. Mirror TTA needs `mirror_channel_permutation (:296)` because flipping the volume also permutes which offset each channel means.

This is the most interesting method in the repo for anyone thinking about topology, because it is the one that makes topology a *training signal* rather than only an evaluation metric.

7. Losses

All in `losses/common.py`:

Key	What
<code>dice</code>	plain soft Dice after sigmoid
<code>dice_ce</code>	Dice + BCE (1 channel) or Dice + CE (multi-channel)
<code>multiclass_dice</code>	softmax Dice averaged over classes — CAS-Net
<code>weighted_sim</code>	ImageCAS Eq. 2, $1 - (TP+s)/(a\cdot pred + (1-a)\cdot gt + s)$; $a=0.01$ makes false negatives $\sim 100\times$ costlier than false positives
<code>deep_supervision_dice</code>	any base loss averaged over U-Net++’s head list

ade_htl	$\lambda_1 \cdot \text{Dice} + (1 - \lambda_1) \cdot \text{CE} + \lambda_2 \cdot (\text{weighted-Hausdorff} + \text{MSE})$ across the four heads
---------	--

Two internal classes are not registered but matter: `ChannelwiseDiceLoss (:25)`, used for the 27 connectivity channels because a pooled Dice normalises by total positive count and lets the dense axis-aligned offsets drown out the sparse diagonal ones; and `WeightedHausdorffLoss (:121)`, a probability-map-to-point-set distance computed against a precomputed EDT rather than a full pairwise distance matrix.

One comment in `ADEHTLLoss` is worth reading in full (:200-206): the CE term is *deliberately unweighted*, because inverse-frequency weighting puts the optimal probability at exactly 0.5 — precisely the threshold the connectivity fusion cuts at, leaving the decision no margin.

Note for anyone looking for a topology-aware loss: there isn't one. `cl_dice` exists only as an evaluation metric (`utils/metrics.py:71`); the nnU-Net+`clDice` benchmark row is trained entirely outside this repo. Adding a `clDice` loss is a clean fit for the documented extension point.

8. Metrics and evaluation

The eleven headline metrics

`utils/metrics.py`, `METRIC_REGISTRY` at :252. All computed against the original-resolution GT, so units are physical.

Metric	Meaning
dice	overlap
hd95	symmetric 95th-percentile surface distance, mm
cl_dice	centerline Dice (Shit et al. 2021) — harmonic mean of “how much of the predicted skeleton is inside the GT” and vice versa. The connectivity metric.
centerline_md	mean distance between predicted skeleton and GT centerline points, mm
centerline_hd95	95th-percentile version of the same
betti_error_1	$ \Delta \text{ connected components} $ — fragmentation
betti_error_2	$ \Delta \text{ loops} $ — spurious cycles
volume_{pred,gt,md,bias}_ml	lumen volume, absolute difference, and signed bias

`_betti_numbers (:4)` takes b_0 from 26-connected labelling and approximates $b_1 \approx \max(0, b_0 - \text{Euler characteristic})$, which is valid for tubular structures with no enclosed voids.

The two breakdowns

This is where the benchmark gets genuinely informative, and it is worth understanding because it is the part most likely to be reused.

Scan-level strata come straight from Descriptors.xlsx — every column in evaluation.stratify_by (Image Quality, Dominance, Disease) is split by its values and the full metric set recomputed within each.

Geometric strata come from a precompute:

```
python -m utils.precompute_centerline_samples -c configs/<method>.json --split test
```

For each scan this walks the GT centerline tree and samples vertices, recording per sample point: dist_mm (arc length from the ostium), segment_name, diameter_mm (from the lumen EDT), and hu (mean attenuation in the local lumen). Evaluation then adds **local_dice** — Dice inside an 8 mm cube centred on that point — and reports it grouped by segment, and by binned distance / attenuation / diameter.

So instead of “method X gets 0.82 Dice”, you get “method X gets 0.9 in the proximal LAD and 0.4 in vessels under 1.5 mm at low attenuation”. That is the point of the paper.

Two design choices in there:

- The local ROI is a **fixed physical size**, not a fixed voxel count — otherwise a 16^3 box would mean 8 mm in one scan and 5 mm in another, and pooled bins would silently mix them.
- Aggregation is **macro**: one mean per scan per bin, then statistics over scans (`_summarise_points`, `evaluate.py:602`). Pooling raw points would let a long, well-opacified tree outweigh a short one.

9. The centerline analysis code

`utils/precompute_centerline_samples.py` is the most reusable piece of geometry in the repo, so it is worth knowing function by function:

Function	Line	What
<code>_read_centerline(path)</code>	:47	points, line cells, and <i>all</i> point-data arrays (segment_label, segment_name, start_points, end_points, branch_points)
<code>_adjacency(cells, n_points)</code>	:91	undirected adjacency over point IDs — the graph
<code>_traverse(points, adj, roots)</code>	:104	multi-source BFS from every ostium at once , returning hop depth and true arc-length dist_mm per vertex
<code>_sample_vertices(...)</code>	:126	every Nth vertex <i>by hop count</i> , plus all endpoints
<code>_radius_field(lumen, spacing)</code>	:153	EDT inside the lumen = local radius in mm
<code>_GTGeometry</code>	:167	cKDTree over lumen voxels; snaps a centerline point onto the mask and reads radius / HU there

BFS runs from all roots simultaneously because a left system with no left main has *separate* LAD and LCx ostia — so a “side” may hold more than one tree, and each vertex measures from whichever ostium reaches it first.

Sampling by hop count rather than arc length means a shared proximal trunk is sampled once however many tips lie beyond it, and sampling stays phase-aligned across a bifurcation.

Elsewhere:

- `utils/io.py:118 load_vtk_centerline` — points + all point arrays.
- `utils/io.py:151 load_vtk_centerline_branches` — **per-branch point arrays in walk order**, from the line cells. Defined but never called; useful if you need ordered polylines rather than a point cloud.
- `utils/io.py:238 centerline_points_to_mask` — rasterise branches to a 1-voxel-thick mask without drawing shortcuts across branch boundaries. Also currently unused.
- `evaluate.py:273 _load_centerline_by_segment` — split points by `segment_label`.
- `models/imagecas_baseline/model.py:27 _extract_skeleton` — skimage 3D thinning, used for predicted masks; `:33 _remove_small_components` — size-filtered labelling.

What is *not* there

No graph library is installed at all (`pyproject.toml:17-27`: `numpy`, `scipy`, `scikit-image`, `scikit-learn`, `SimpleITK`, `vtk`, `tqdm`, `matplotlib`, `openpyxl` — no `networkx`, no `skan`, no `VMTK`). The only graph structure in the codebase is the hand-rolled adjacency list above.

Nothing computes branch decomposition into edges between bifurcations, generation or Strahler order, bifurcation angles, tortuosity, taper, or dominance derived from anatomy. Predicted skeletons are only ever used as **unordered point clouds** — there is no skeleton→graph conversion anywhere.

Also note: `branch_points` is deliberately dropped from the cached `.npz` (`precompute_centerline_samples.py:263`), because bifurcations only land on a sampling boundary by accident and a half-sampled covariate would be misleading. Anything that needs bifurcations re-reads the VTK, as `models/ade_htl/precompute_targets.py:113-117` does.

10. The offline-precompute pattern

Five scripts share one shape, and it is the pattern to copy for anything new:

```
python -m <module> -c configs/<method>.json --split all --workers 8 [--overwrite]
```

`ProcessPoolExecutor` over scan IDs, one `.npz` or `.numpy` per scan written to a directory under `<data_root>`, resume-by-default, config-driven paths.

Script	Produces
<code>utils/offline_resample_images_to_disk.py</code>	the shared 0.5 mm <code>.numpy</code> cache
<code>utils/precompute_centerline_samples.py</code>	<code>centerline_samples/<id>.npz</code> — point covariates
<code>utils/offline_generate_mesh_from_voxel.py</code>	<code>surfaces/<id>.coronary_surface.vtk</code> — marching cubes + Taubin smoothing
<code>models/ade_htl/precompute_ade.py</code>	ADE ROI + 5 chamber distance fields
<code>models/ade_htl/precompute_targets.py</code>	key points + centerline heatmap
<code>models/ade_htl/precompute_dilated_masks.py</code>	<code><id>_dilated.npy</code>
<code>models/imagecas_baseline/stage2_resample_dilated_masks.py</code>	<code>imagecas/<id>.npz</code> cache

Script	Produces
<code>models/imagecas_baseline/stage3_generate_centers.py</code>	centres from the Stage-2 skeleton

11. Running things end to end

Setup

```
pip install -e .
export ImageCAS_X_data_path=/path/to/data
export ImageCAS_X_results_path=/path/to/results
python -m utils.offline_resample_images_to_disk -c configs/cas_net.json --workers 8
```

A single-stage method (CAS-Net, FFR-UNet, Swin UNETR)

```
python -m train -c configs/cas_net.json
python -m inference -c configs/cas_net.json -r <run_dir>
python -m evaluate -c configs/cas_net.json -r <run_dir>
```

ImageCAS (4 training runs + a centre-generation step, then one ensemble inference)

```
python -m models.imagecas_baseline.stage2_resample_dilate -c configs/imagecas_stage2_coarse_dil
python -m train -c configs/imagecas_stage1_coarse.json
python -m train -c configs/imagecas_stage2_coarse_dilated.json
python -m models.imagecas_baseline.stage3_generate_centers \
    -c configs/imagecas_stage2_coarse_dilated.json \
    --coarse-checkpoint <stage2_best.pt> --out-dir <centers_dir> --split all
# set data.params.centers_dir in each stage-3 config, then one run each:
python -m train -c configs/imagecas_stage3_patch_{16,32,64}.json
# set all 5 model.*_checkpoint fields in imagecas_inference.json, then:
python -m inference -c configs/imagecas_inference.json -r <run_dir>
```

ADE-HTL (a coarse localisation pass, then two precomputes, then the main network)

```
python -m models.ade_htl.precompute_dilated_masks -c configs/ade_htl_stage1_coarse.json --split
python -m train -c configs/ade_htl_stage1_coarse.json
python -m inference -c configs/ade_htl_stage1_coarse.json -r <run_dir> --split train # then va
python -m models.ade_htl.precompute_ade -c configs/ade_htl.json --coarse-run <run_dir> --sp
python -m models.ade_htl.precompute_targets -c configs/ade_htl.json --split all
python -m train -c configs/ade_htl.json
```

Useful side tools

```
python -m utils.verify_data loaders -c configs/<method>.json # PNG sanity check of the real
python -m inference -c ... -r ... --computational_analysis # timing table
python -m models.nnunet.data_prep -c ... --nnunet-data-folder <dir>
```

`utils/display_in_slicer.py` is a paste-into-3D-Slicer script for viewing segments, center-lines and method comparisons.

12. Things to know before you change anything

- **preprocessing.steps is partly a description, not an instruction.** Steps already baked into an offline cache are skipped at runtime (`dataloading/base_dataset.py:127`). If you add a preprocessing step, check whether it needs to go into the cache builder too.
- **Lists in configs replace, they don't merge.** Declaring `preprocessing.steps` in a method config throws away `pipeline.json`'s list entirely.
- **Centerline VTK cell order carries no direction.** Always traverse from `start_points`.
- **Models must return raw logits in a dict.** Any activation belongs in postprocessing, or the uniform threshold-at-0.5 path breaks.
- **train.py casts logits to fp32 before the loss.** If you add a loss that sums over many voxels, do not undo this.
- **Augmentation can invalidate precomputed targets.** ADE-HTL derives its connectivity target *after* augmentation for exactly this reason; anything orientation-dependent must do the same.
- **Postprocessing keeps components over 100 voxels rather than the largest one,** so the left and right trees both survive without hardcoding how many components to expect.
- **Known inconsistency:** `README.md:111` and the ImageCAS-X paper itself (Data Records, Fig. 3) both document `surfaces/<id>.coronary_mesh.vtk` as the canonical filename, but `utils/offline_generate_mesh_from_voxel.py:340` writes `.coronary_surface.vtk`. Since the published data record confirms the intended name, this is a real bug in the local script, not just a stale doc — worth fixing if you rely on this script rather than the released Zenodo files (which presumably use the correct name).
- **models.nnunet is intentionally not imported** by `models/registry.py`, so `configs/nnunet.json` would fail in `build_model` — it is an evaluate-only config.